



哈爾濱工業大學 (深圳)
HARBIN INSTITUTE OF TECHNOLOGY

实验报告

开课学期: 2025 春季
课程名称: 人工智能 (实验)
实验名称: 人工智能实验 2: 深度学习
实验性质: 综合设计型
实验学时: 4 地点: T2612
学生姓名: 刘睿
学生学号: 220110720
学生班级: 计科 7 班
授课教师: _____

实验与创新实践教育中心制

2025 年 4 月

说明:

- 1、有修改的代码源文件需提交，数据集和模型参数不要提交。
- 2、可以自行用 markdown 格式最后导出 pdf，排版要一致，封面只需体现对应的信息。
- 3、训练有准确率的要求，注意看指导书；模板中的说明文字可以删除。

1、Mindspore

(1) 简单说明做了哪些超参数调整，为什么调整这些参数

本次仅修改了优化器，将原本的 SGD 更换为 Adam，并设置了如下参数：

`learning_rate = 1e-2, weight_decay = 1e-2`

(上传的代码用于演示微调，因此 `learning_rate` 设置为 `1e-4`)

由于 MNIST 是一个非常简单的数据集，模型在不进行 BatchNorm 和 Dropout 等结构性正则化的前提下，仍可轻松达到 95% 以上的准确率。因此，为了更好地适应这种情况，选择使用 Adam 优化器以增强模型的自适应能力，同时设置 `weight_decay=1e-2` 以有效降低过拟合风险

```
# Instantiate loss function and optimizer
loss_fn = nn.CrossEntropyLoss()
lr = 1e-4
optimizer = nn.Adam(model.trainable_params(), learning_rate=lr, weight_decay=1e-2)
```

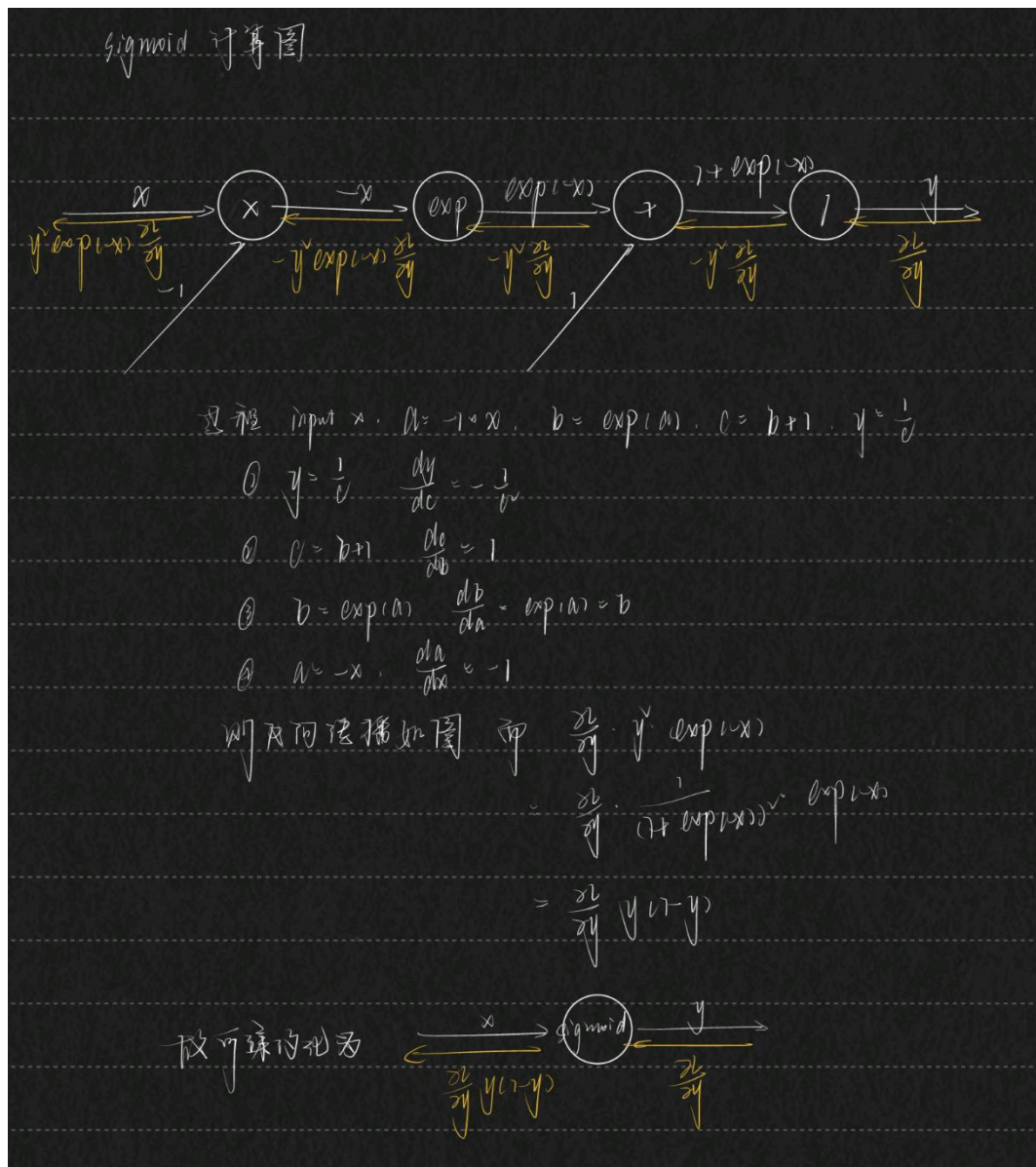
此外，添加了一个简单的早停机制，用于在模型训练过程中避免冗余训练

```
for t in range(epochs):
    print(f"Epoch {t+1}\n-----")
    train(model, train_dataset)
    acc = test(model, test_dataset, loss_fn)

    if acc < last_acc + 0.005:
        print(f'Early stop at {t} epoch')
        break
```

2、反向传播和全连接网络

(1) sigmoid 函数计算图和反向传播推导过程，计算图需清楚清晰



(2) train_fcnet.py 训练做了哪些超参数调整

原始的 $lr=0.01$, $epoch=10$, 难以训练速度太慢迭代次数太少难以达到 90% 正确率的标准, 于是将 $lr=0.02$, $epoch=30$, 加速训练并增加迭代次数。最终可以达到 91% 的正确率

```
In [10]: !python train_fcnet.py

Downloading MNIST dataset from https://github.com/mnielsen/neural-networks-and-deep-learning/raw/master/data/mnist.pkl.gz...
Download finished!
train size 50000
epoch 0 train acc: 0.1136, test acc: 0.1135
epoch 1 train acc: 0.1136, test acc: 0.1135
epoch 2 train acc: 0.3389, test acc: 0.3386
epoch 3 train acc: 0.4924, test acc: 0.4969
epoch 4 train acc: 0.6333, test acc: 0.6440
epoch 5 train acc: 0.7219, test acc: 0.7325
epoch 6 train acc: 0.7812, test acc: 0.7886
epoch 7 train acc: 0.8116, test acc: 0.8184
epoch 8 train acc: 0.8342, test acc: 0.8405
epoch 9 train acc: 0.8467, test acc: 0.8528
epoch 10 train acc: 0.8569, test acc: 0.8623
epoch 11 train acc: 0.8663, test acc: 0.8725
epoch 12 train acc: 0.8744, test acc: 0.8793
epoch 13 train acc: 0.8794, test acc: 0.8851
epoch 14 train acc: 0.8847, test acc: 0.8895
epoch 15 train acc: 0.8885, test acc: 0.8932
epoch 16 train acc: 0.8916, test acc: 0.8959
epoch 17 train acc: 0.8940, test acc: 0.8975
epoch 18 train acc: 0.8972, test acc: 0.9010
epoch 19 train acc: 0.8996, test acc: 0.9033
epoch 20 train acc: 0.9019, test acc: 0.9048
epoch 21 train acc: 0.9036, test acc: 0.9067
epoch 22 train acc: 0.9047, test acc: 0.9081
epoch 23 train acc: 0.9061, test acc: 0.9093
epoch 24 train acc: 0.9075, test acc: 0.9103
epoch 25 train acc: 0.9088, test acc: 0.9111
epoch 26 train acc: 0.9098, test acc: 0.9124
epoch 27 train acc: 0.9108, test acc: 0.9139
epoch 28 train acc: 0.9120, test acc: 0.9143
epoch 29 train acc: 0.9126, test acc: 0.9152
gradient compute time: 33.93762278556824 s
predict label: [7 2 1 0] true label [7 2 1 0]
Figure(640x480)
```

3、卷积

(1) util.py/conv_forward_naive 代码截图、train_convnet.py 代码截图，训练日志截图

```
109     def conv_forward_naive(x, w, b, conv_param):
137
138         stride = conv_param['stride']
139         pad = conv_param['pad']
140
141         N, C, H, W = x.shape
142         F, _, HH, WW = w.shape
143
144         x_p = np.pad(x, ((0,0), (0,0), (pad,pad), (pad,pad)), mode='constant')
145
146         H_out = 1 + (H + 2 * pad - HH) // stride
147         W_out = 1 + (W + 2 * pad - WW) // stride
148         out = np.zeros((N, F, H_out, W_out))
149
150         x_windows = np.lib.stride_tricks.sliding_window_view(x_p, (HH, WW), axis=(2,3))
151         x_windows = x_windows[:, :, ::stride, ::stride, :, :]
152
153         for n in range(N):
154             for f in range(F):
155                 conv = np.sum(x_windows[n] * w[f][:, None, None, :, :], axis=(0,3,4))
156                 out[n, f] = conv + b[f]
157
158         cache = (x_p, w, b, conv_param)
159         return out, cache
```

train_convnet.py 对比 train_fcnet.py 有两处必要的修改点

1: 将数据处理成合适卷积网络输入的格式（不展平，将灰度图通道的维度添加进来），另外就是实例化网络的时候修改一下实例化的对象就行

```
12     # 读入数据
13     (x_train, t_train), (x_test, t_test) = load_mnist(flatten=False)
14
15     if x_train.ndim == 3:
16         x_train = x_train[:, np.newaxis, :, :]
17         x_test = x_test[:, np.newaxis, :, :]
18
19     network = SimpleConvNet()
```

2: 对应 SimpleConvNet() 的实现修改一下需要更新的参数字典

```
46         # 使用梯度下降法更新权重
47         for key in ('w1', 'b1', 'w2', 'b2', 'w3', 'b3'):
48             network.params[key] -= learning_rate * grad[key]
```

3: 此外, 50000 的训练集实在是太慢了, 所以自己 shuffle 了一下训练集并选择了其中 10000 用于训练

```
15     shuffle_index = np.random.permutation(len(x_train))
16     x_train = x_train[shuffle_index]
17     t_train = t_train[shuffle_index]
18
19     x_train = x_train[:10000]
20     t_train = t_train[:10000]
```

训练日志:

```
In [5]: !cd src && python train_convnet.py

Downloading MNIST dataset from https://github.com/mnielsen/neural-networks-and-deep-learning/raw/master/data/mnist.pkl.gz...
Download finished!
train size 10000
epoch 0 train acc: 0.1085, test acc: 0.1117
epoch 1 train acc: 0.3552, test acc: 0.3643
epoch 2 train acc: 0.6235, test acc: 0.6262
epoch 3 train acc: 0.7442, test acc: 0.7522
epoch 4 train acc: 0.7946, test acc: 0.8117
epoch 5 train acc: 0.8226, test acc: 0.8402
epoch 6 train acc: 0.8430, test acc: 0.8530
epoch 7 train acc: 0.8601, test acc: 0.8694
epoch 8 train acc: 0.8683, test acc: 0.8763
epoch 9 train acc: 0.8723, test acc: 0.8810
epoch 10 train acc: 0.8800, test acc: 0.8868
epoch 11 train acc: 0.8862, test acc: 0.8917
epoch 12 train acc: 0.8924, test acc: 0.8945
epoch 13 train acc: 0.8930, test acc: 0.8981
epoch 14 train acc: 0.8954, test acc: 0.8991
epoch 15 train acc: 0.8976, test acc: 0.9030
early stop at epoch 15 train acc: 0.8976, test acc: 0.9030
gradient compute time: 404.7153787612915 s
```

(2) 若做选做, 介绍下实现方案和实验结果

4、PyTorch

(1) 自定义模型代码截图、训练日志截图，简单介绍下为什么这么设计，训练过程中做了哪些超参数调整以及为什么这么调整

自定义模型代码如下，因为隔壁计算机视觉课程作业有一个 CIFAR-10 上的任务，课程要求的模型结构是二层卷积+三层全连接，但拼尽全力调参无法突破 70%正确率，索性这里的就加深一层卷积并增大维度，配合两层全连接获得输出

```
class CNN(nn.Module):
    def __init__(self):
        super(CNN, self).__init__()

        self.features = nn.Sequential(
            nn.Conv2d(3, 32, 3, padding=1),
            nn.ReLU(),
            nn.Conv2d(32, 64, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2, 2),

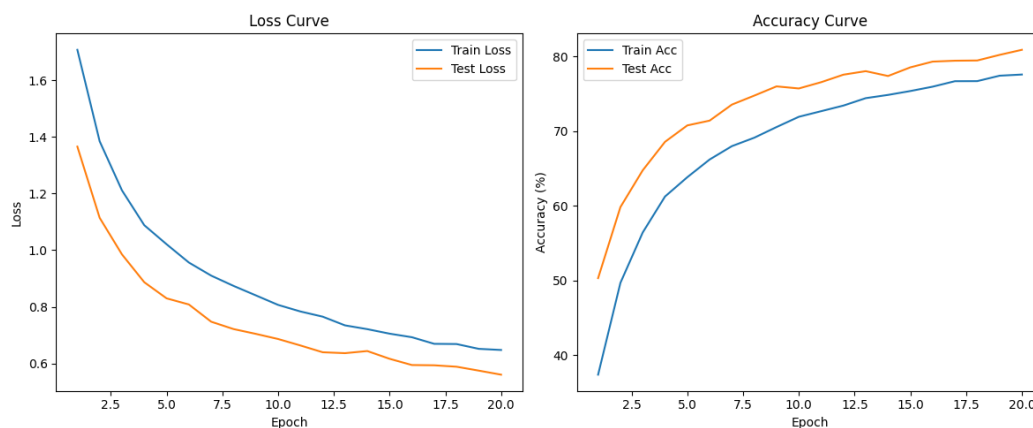
            nn.Conv2d(64, 128, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2, 2),
        )
        self.classifier = nn.Sequential(
            nn.Linear(8*8*128, 256),
            nn.ReLU(),
            nn.Dropout(0.5),
            nn.Linear(256, 10)
        )

    def forward(self, x):
        x = self.features(x)
        x = x.view(x.size(0), -1)
        x = self.classifier(x)
        return x
```

设置的超参数 epoch=20，从训练日志来看其实可以多训练一些 epoch 来获得更好的模型，不过自定义的小模型在 CIFAR-10 上有 80% 左右的 acc 差不多了，就没有进一步训练
优化器选的 Adam，设置 lr = 1e-e3

训练日志如下：

```
Epoch [1/20] - Train Loss: 1.7076, Train Acc: 37.38% | Test Loss: 1.3658, Test Acc: 50.32%
Epoch [2/20] - Train Loss: 1.3846, Train Acc: 49.69% | Test Loss: 1.1143, Test Acc: 59.84%
Epoch [3/20] - Train Loss: 1.2112, Train Acc: 56.44% | Test Loss: 0.9852, Test Acc: 64.77%
Epoch [4/20] - Train Loss: 1.0882, Train Acc: 61.26% | Test Loss: 0.8869, Test Acc: 68.59%
Epoch [5/20] - Train Loss: 1.0210, Train Acc: 63.85% | Test Loss: 0.8300, Test Acc: 70.79%
Epoch [6/20] - Train Loss: 0.9564, Train Acc: 66.22% | Test Loss: 0.8083, Test Acc: 71.43%
Epoch [7/20] - Train Loss: 0.9102, Train Acc: 68.01% | Test Loss: 0.7475, Test Acc: 73.58%
Epoch [8/20] - Train Loss: 0.8740, Train Acc: 69.14% | Test Loss: 0.7215, Test Acc: 74.78%
Epoch [9/20] - Train Loss: 0.8405, Train Acc: 70.57% | Test Loss: 0.7045, Test Acc: 76.02%
Epoch [10/20] - Train Loss: 0.8068, Train Acc: 71.94% | Test Loss: 0.6864, Test Acc: 75.74%
Epoch [11/20] - Train Loss: 0.7836, Train Acc: 72.69% | Test Loss: 0.6636, Test Acc: 76.58%
Epoch [12/20] - Train Loss: 0.7653, Train Acc: 73.45% | Test Loss: 0.6396, Test Acc: 77.59%
Epoch [13/20] - Train Loss: 0.7345, Train Acc: 74.44% | Test Loss: 0.6364, Test Acc: 78.06%
Epoch [14/20] - Train Loss: 0.7212, Train Acc: 74.88% | Test Loss: 0.6439, Test Acc: 77.41%
Epoch [15/20] - Train Loss: 0.7052, Train Acc: 75.40% | Test Loss: 0.6165, Test Acc: 78.57%
Epoch [16/20] - Train Loss: 0.6926, Train Acc: 75.98% | Test Loss: 0.5945, Test Acc: 79.34%
Epoch [17/20] - Train Loss: 0.6693, Train Acc: 76.72% | Test Loss: 0.5936, Test Acc: 79.46%
Epoch [18/20] - Train Loss: 0.6686, Train Acc: 76.72% | Test Loss: 0.5885, Test Acc: 79.49%
Epoch [19/20] - Train Loss: 0.6516, Train Acc: 77.45% | Test Loss: 0.5747, Test Acc: 80.24%
Epoch [20/20] - Train Loss: 0.6476, Train Acc: 77.61% | Test Loss: 0.5604, Test Acc: 80.92%
Finished Training
```



很神奇啊，一直在验证集上表现更好

(2) 分别给出 5.pytorch.ipynb 中用到的两层全连接网络、三层卷积网络、自定义网络的参数量，要求有计算过程，以及使用 `model.parameters()`、`model.state_dict()` 或者 `pytorch-summary` 等查看模型参数的输出结果截图

Two layer FC (3072x3072, 4000, 10)

3072 $\rightarrow$ 4000 $\rightarrow$ 10

fc1 的权重参数: $4000 \times 3072 / 2 = 12,288,000$

fc1 的偏置: 4000

fc2 的权重: $10 \times 4000 = 40,000$

fc2 的偏置: 10

合计: 12,328,010 个参数

```
fc1.weight: torch.Size([4000, 3072])
```

```
fc1.bias: torch.Size([4000])
```

```
fc2.weight: torch.Size([10, 4000])
```

```
fc2.bias: torch.Size([10])
```

Three Layer Conv Net 13, 32, 16, 10

conv1 的权重: $32 \times 3 \times 5 \times 5 = 2400$

conv1 的偏置: 32

conv2 的权重: $16 \times 32 \times 3 \times 3 = 4608$

conv2 的偏置: 16

fc 的权重: $10 \times 16384 = 163840$

fc 的偏置: 10

总计: 170,906 个参数

```
conv1.weight: torch.Size([32, 3, 5, 5])
conv1.bias: torch.Size([32])
conv2.weight: torch.Size([16, 32, 3, 3])
conv2.bias: torch.Size([16])
fc.weight: torch.Size([10, 16384])
fc.bias: torch.Size([10])
```

CNN ()

conv1 的权重

$$32 \times 3 \times 3 \times 3 = 864$$

偏置

32

conv2 的权重

$$64 \times 32 \times 3 \times 3 = 18432$$

偏置

64

conv3 的权重

$$128 \times 64 \times 3 \times 3 = 73728$$

偏置

128

fc1 的权重

$$8192 \times 256 = 2097152$$

偏置

256

fc 的权重

$$256 \times 10 = 2560$$

偏置

10

合计 2,119,498 个参数

5、实验中碰到的问题、总结、建议等

5.1 碰到的问题

解决了的问题：

在 train_convnet 的任务中，一开始我想得很简单，把 train_fcnet 的代码直接拷贝过来，改一下参数字典的更新，改一下数据的输入格式就直接用的但是实际遇到了两个问题

1. 训练速度过慢，等待时间太久
2. 训练效果不加，仅调整学习率也没用，总是过早收敛到某个非常差的局部最优

```
15.6s    23    epoch 0 train acc: 0.1185, test acc: 0.1275
19.5s    24    epoch 1 train acc: 0.1065, test acc: 0.1235
23.6s    25    epoch 2 train acc: 0.0910, test acc: 0.1005
27.8s    26    epoch 3 train acc: 0.1105, test acc: 0.1170
31.9s    27    epoch 4 train acc: 0.1100, test acc: 0.1170
36.0s    28    epoch 5 train acc: 0.1000, test acc: 0.0940
40.2s    29    epoch 6 train acc: 0.1040, test acc: 0.1010
44.3s    30    epoch 7 train acc: 0.1130, test acc: 0.1205
48.5s    31    epoch 8 train acc: 0.0895, test acc: 0.0985
52.5s    32    epoch 9 train acc: 0.1025, test acc: 0.0985
56.7s    33    epoch 10 train acc: 0.0890, test acc: 0.1005
60.8s    34    epoch 11 train acc: 0.0905, test acc: 0.1050
64.7s    35    epoch 12 train acc: 0.1070, test acc: 0.1215
68.6s    36    epoch 13 train acc: 0.1015, test acc: 0.0955
72.6s    37    epoch 14 train acc: 0.1030, test acc: 0.0985
76.4s    38    epoch 15 train acc: 0.1115, test acc: 0.1025
80.4s    39    epoch 16 train acc: 0.1040, test acc: 0.1000
84.4s    40    epoch 17 train acc: 0.1120, test acc: 0.1025
88.3s    41    epoch 18 train acc: 0.1120, test acc: 0.1025
92.3s    42    epoch 19 train acc: 0.1120, test acc: 0.1025
96.3s    43    epoch 20 train acc: 0.1120, test acc: 0.1025
```

（减小训练集到 500 的一次测试）

首先对于训练过慢的问题，毕竟是直接 numpy 上搭的神经网络，不可避免很慢，所以只好采取减少训练集大小的方式。原训练集大小为 50000，其实减小到 10000 以下就有还能看的速度了，至少不用完全看不到结局

对于训练效果不佳的问题，我尝试调了模型卷积核数量、通道数量和学习率，基本都没什么成效，于是我关注到了模型中还有一个参数 weight_init_std，这个参数决定了模型参数初始化时候的 std，这个默认是 1e-2，但是太小的话可能权重什么的差异不大，模型无法被真正激活，于是我把这个参数调整为了 1e-1，于是立马就突破了训练不佳的困境，可以在最开始几个 epoch 就达到 60% 的正确率

```
13.5s    21    train size 10000
40.4s    22    epoch 0 train acc: 0.1085, test acc: 0.1117
109.0s   23    epoch 1 train acc: 0.3552, test acc: 0.3643
177.3s   24    epoch 2 train acc: 0.6235, test acc: 0.6262
```

未解决的问题:

在 windows 本机上利用 ollama 部署 deepseek-r1 的时候，一开始我都是采用默认的步骤，正确配置好了 deepseek-r1:1.5b 的模型，并可以正确使用。但是后来我嫌弃太占 C 盘，于是寻找方法，利用 cmd 中执行 ollamasetup.exe 并设置 /DIR 参数成功把 ollama 安装到 D 盘，设置好 OLLAMA_MODELS 的环境变量也把模型迁移到了 D 盘，然后！

```
PS C:\Users\hakur> ollama run deepseek-r1:1.5b >>> 你好
Error: POST predict: Post "http://127.0.0.1:49306/completion": read
tcp 127.0.0.1:49308->127.0.0.1:49306: wsarecv: An existing connection
was forcibly closed by the remote host.
```

就开始这样报错，完全不知道怎么回事

看日志，发现 GGML_ASSERT(tensor->op == GGML_OP_UNARY) failed

但是我也不知道怎么处理，只好一切重来，无数关机开机设置环境变量之后，即使全按默认的方式来装，也无法解决这个报错了

至此，成为未解之谜，我也没太多时间处理这个问题了

5.2 总结、建议

验收太快了